

Whack-A-Mole – Technical Documentation

1. Final OOP Design and Justification

The Whack-A-Mole game is designed using a modular OOP structure consisting of three core packages: model, engine, and ui. Each package handles a dedicated responsibility, improving maintainability, clarity, and future scalability.

1.1 Model Package

• HoleOccupant (Abstract Class)

- Represents any entity that appears inside a hole during gameplay.
Attributes:
- icon : ImageIcon
- scoreValue : int

Subclasses:

- Mole – gives positive score
- Bomb – gives negative score
- BonusMole – gives high positive score

Justification:

Using inheritance allows the GameEngine to treat all hole occupants uniformly while enabling each subclass to define its unique scoring behavior. This makes the system flexible and easy to extend with future occupant types.

• PlayerScore

- A serializable class that stores:
- name : String
- score : int
- This class supports persistent tracking of high scores.

1.2 Engine Package

- GameEngine (implements Runnable)
- Responsible for all core game logic:
- Timer countdown
- Spawning/removing moles
- Handling user clicks
- Updating scores

- Determining game end

Justification:

Separating game logic from the UI maintains responsiveness. Running GameEngine on a separate thread prevents UI lag and supports smooth gameplay.

• **HighScoreManager**

Handles saving and loading player scores using a file.

Provides:

- `load() : int` – returns highest score
- `save(score : int)` – saves updated score

Justification:

Encapsulates file operations, keeping the rest of the application free from low-level I/O concerns. Enhances modularity and simplifies debugging.

• **HighScoreException (Custom Exception)**

Thrown when high-score loading/saving fails.

Provides clear error information without exposing internal I/O details.

• **InvalidGameStateException (Custom Exception)**

Indicates inconsistent or unexpected game states (optional use).

Useful for catching logical or programming errors.

1.3 UI Package

• **GameGrid (extends JFrame)**

The main GUI window displaying:

- Score label
- Timer
- High score
- 4×4 hole grid

It communicates with the GameEngine to reflect real-time game state.

• **HoleButton (extends JButton)**

Represents a single hole on the grid.

Stores the index of its position and displays the appropriate occupant icon.

• **WhackAMoleApp**

The entry point of the program.

Loads image resources, initializes the high-score system, and launches the game UI.

2. Concurrency Strategy

The game uses a multi-threaded architecture to maintain smooth gameplay and responsive UI. Two primary threads are involved:

2.1 Swing Event Dispatch Thread (EDT)

Handles all user interaction and UI updates:

- Button clicks
- Screen rendering
- Label updates

Swing requires that all UI modifications occur on the EDT to prevent thread-safety issues.

2.2 GameEngine Thread

Runs the game loop independently:

- Controls timing
- Spawns/removes occupants
- Updates internal game state
- Allows the UI to remain responsive even during heavy gameplay logic.

2.3 Safe Thread Communication

The GameEngine never touches UI components directly.

Instead, it uses:

```
SwingUtilities.invokeLater(() -> {  
    // Safe UI update});
```

This ensures:

- No race conditions
- No cross-thread UI access
- Smooth rendering and gameplay

2.4 Thread Shutdown

When the window is closed:

The engine thread receives `interrupt()`

It periodically checks `Thread.interrupted()`

It exits gracefully

This avoids:

- Zombie threads
- Memory leaks
- Crashes on exit

3. Custom Exceptions and Rationale

3.1 HighScoreException

- Thrown when score data cannot be loaded or saved properly.

Rationale:

- Provides meaningful error messages
- Separates persistence concerns from game logic
- Prevents the UI from crashing due to file errors

3.2 InvalidGameStateException

- Signals logically inconsistent states (e.g., invalid grid dimensions).

Rationale:

- Ensures internal stability
- Helps programmers catch logic bugs
- Maintains clean error-handling flow

4. Summary

- The Whack-A-Mole project is built with clean, modular OOP principles. Key strengths include:
 - Inheritance & Polymorphism for flexible occupant behavior
 - Composition for clear object relationships
 - Separation of Concerns between UI, logic, and data management
 - Multithreading for responsive gameplay
 - Custom Exceptions for reliable and readable error management
 - Persistent High-Score Tracking for enhanced user experience
- The result is a robust, maintainable, and extendable game architecture.